

Department of CSE (AI & ML)
ACE Engineering College



**AI307PC: OBJECT ORIENTED
PROGRAMMING USING JAVA LAB**

LAB MANUAL

Course Objectives:

1. To write programs using abstract classes.
2. To write programs for solving real world problems using the java collection framework.
3. To write multithreaded programs.
4. To write GUI programs using swing controls in Java.
5. To introduce java compiler and eclipse platform.
6. To impart hands-on experience with java programming.

Course Outcomes:

1. Able to write programs for solving real world problems using the java collection framework.
2. Able to write programs using abstract classes.
3. Able to write multithreaded programs.
4. Able to write GUI programs using swing controls in Java.

Contents

1	Eclipse IDE Familiarization	4
2	Simple Calculator using GridLayout	6
3	CRUD Operations using JDBC	8
4	Java Collections Framework	10
5	Integer Division with Exception Handling	12
6	Multithreaded Application	14
7	ArrayList Insertion and Deletion	16
8	Traffic Light Simulation	18
9	Abstract Class - Shape Hierarchy	20
10	Displaying a Table from a Text File using GridLayout	22
11	Handling Mouse Events using Adapter Classes	24
12	Contact Lookup using Hash Table	26
13	Producer-Consumer Problem	28
14	Recursively Listing Files in a Directory	30

Experiment 1 Eclipse IDE Familiarization

1. Question

Use Eclipse or Net Bean platform and acquaint yourself with the various menus. Create a test project, add a test class, and run it. See how you can use auto suggestions, auto fill. Try code formatter and code refactoring like renaming variables, methods, and classes. Try debug step by step with a small program of about 10 to 15 lines which contains at least one if else condition and a for loop.

2. AIM

To familiarize with the Eclipse IDE environment (project creation, auto-suggestion, formatting, refactoring, and debugging) using a simple Java program.

3. Code

```
1 public class NumberCheck {
2     public static void main(String[] args) {
3         int n = 10;
4         int sum = 0;
5
6         for (int i = 1; i <= n; i++) {
7             if (i % 2 == 0) {
8                 System.out.println(i + " is Even");
9                 sum = sum + i;
10            } else {
11                System.out.println(i + " is Odd");
12            }
13        }
14        System.out.println("Sum of even numbers from 1 to " + n + " = " +
15        sum);
16    }
17 }
```

4. Output

```
1 1 is Odd
2 2 is Even
3 3 is Odd
4 4 is Even
5 5 is Odd
6 6 is Even
7 7 is Odd
8 8 is Even
9 9 is Odd
10 10 is Even
11 Sum of even numbers from 1 to 10 = 30
```

5. Viva Questions

1. What is the difference between JDK, JRE, and JVM?
2. What is the purpose of a breakpoint in debugging?

3. How does Eclipse auto-suggestion (Content Assist) help programmers?
4. What is code refactoring? Give one example.
5. What is the difference between a workspace and a project in Eclipse?

Experiment 2 Simple Calculator using GridLayout

1. Question

Write a Java program that works as a simple calculator. Use a grid layout to arrange buttons for the digits and for the +, -, *, % operations. Add a text field to display the result. Handle any possible exceptions like divided by zero.

2. AIM

To design a simple GUI-based calculator using Swing GridLayout and handle divide-by-zero exceptions.

3. Code

```
1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 public class SimpleCalculator extends JFrame implements ActionListener {
6     JTextField display;
7     String operator = "";
8     double num1 = 0, result = 0;
9
10    SimpleCalculator() {
11        setTitle("Simple Calculator");
12        display = new JTextField();
13        display.setEditable(false);
14
15        JPanel panel = new JPanel();
16        panel.setLayout(new GridLayout(4, 4, 5, 5));
17
18        String[] buttons = {
19            "7", "8", "9", "/",
20            "4", "5", "6", "*",
21            "1", "2", "3", "-",
22            "0", "=", "%", "+"
23        };
24
25        for (String b : buttons) {
26            JButton btn = new JButton(b);
27            btn.addActionListener(this);
28            panel.add(btn);
29        }
30
31        add(display, BorderLayout.NORTH);
32        add(panel, BorderLayout.CENTER);
33
34        setSize(300, 300);
35        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
36        setVisible(true);
37    }
38
39    public void actionPerformed(ActionEvent e) {
40        String cmd = e.getActionCommand();
41        try {
42            if (cmd.charAt(0) >= '0' && cmd.charAt(0) <= '9') {
```

```

43         display.setText(display.getText() + cmd);
44     } else if (cmd.equals("=")) {
45         double num2 = Double.parseDouble(display.getText());
46         switch (operator) {
47             case "+": result = num1 + num2; break;
48             case "-": result = num1 - num2; break;
49             case "*": result = num1 * num2; break;
50             case "/":
51                 if (num2 == 0) throw new ArithmeticException("Divide
by zero");
52                 result = num1 / num2;
53                 break;
54             case "%": result = num1 % num2; break;
55         }
56         display.setText(String.valueOf(result));
57     } else {
58         num1 = Double.parseDouble(display.getText());
59         operator = cmd;
60         display.setText("");
61     }
62 } catch (ArithmeticException ex) {
63     JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
64 ;
65     display.setText("");
66 } catch (NumberFormatException ex) {
67     JOptionPane.showMessageDialog(this, "Invalid Input");
68     display.setText("");
69 }
70
71 public static void main(String[] args) {
72     new SimpleCalculator();
73 }
74 }

```

4. Output

```

1 A calculator window opens with digit buttons (0-9) and operators (+, -, *,
  /, %).
2 On clicking "5", "+", "3", "=" -> the text field displays 8.0
3 On clicking "5", "/", "0", "=" -> a dialog box shows "Error: Divide by zero"

```

5. Viva Questions

1. What is GridLayout, and how does it arrange components?
2. What is the difference between ActionListener and(ActionEvent)?
3. How do you handle exceptions in a GUI application?
4. What is the difference between checked and unchecked exceptions?
5. What is the role of getActionCommand() in event handling?

Experiment 3 CRUD Operations using JDBC

1. Question

Implement CRUD operations in Java using JDBC.

2. AIM

To perform Create, Read, Update, and Delete operations on a MySQL database table using JDBC.

3. Code

```
1 import java.sql.*;
2 import java.util.Scanner;
3
4 public class JBCCrud {
5     static final String URL = "jdbc:mysql://localhost:3306/testdb";
6     static final String USER = "root";
7     static final String PASS = "password";
8
9     public static void main(String[] args) throws Exception {
10         Class.forName("com.mysql.cj.jdbc.Driver");
11         Connection con = DriverManager.getConnection(URL, USER, PASS);
12         Statement stmt = con.createStatement();
13
14         stmt.executeUpdate("CREATE TABLE IF NOT EXISTS student(id INT
15 PRIMARY KEY, name VARCHAR(50), marks INT)");
16
17         Scanner sc = new Scanner(System.in);
18         int choice;
19         do {
20             System.out.println("\n1.Insert 2.View 3.Update 4.Delete 5.Exit")
21 ;
22             choice = sc.nextInt();
23             switch (choice) {
24                 case 1:
25                     System.out.print("Enter ID, Name, Marks: ");
26                     int id = sc.nextInt();
27                     String name = sc.next();
28                     int marks = sc.nextInt();
29                     stmt.executeUpdate("INSERT INTO student VALUES(" + id +
30 ",'" + name + "'," + marks + ")");
31                     break;
32                 case 2:
33                     ResultSet rs = stmt.executeQuery("SELECT * FROM student"
34 );
35                     while (rs.next()) {
36                         System.out.println(rs.getInt("id") + " " + rs.
37 getString("name") + " " + rs.getInt("marks"));
38                     }
39                     break;
40                 case 3:
41                     System.out.print("Enter ID to update marks: ");
42                     int uid = sc.nextInt();
43                     System.out.print("New Marks: ");
44                     int newMarks = sc.nextInt();
```

```
40         stmt.executeUpdate("UPDATE student SET marks=" +
    newMarks + " WHERE id=" + uid);
41         break;
42     case 4:
43         System.out.print("Enter ID to delete: ");
44         int did = sc.nextInt();
45         stmt.executeUpdate("DELETE FROM student WHERE id=" + did
    );
46         break;
47     }
48     } while (choice != 5);
49
50     con.close();
51 }
52 }
```

4. Output

```
1 1.Insert 2.View 3.Update 4.Delete 5.Exit
2 1
3 Enter ID, Name, Marks: 101 John 85
4
5 1.Insert 2.View 3.Update 4.Delete 5.Exit
6 2
7 101 John 85
8
9 1.Insert 2.View 3.Update 4.Delete 5.Exit
10 5
```

5. Viva Questions

1. What is JDBC, and why is it used?
2. What are the steps to connect Java with a database?
3. What is the difference between Statement and PreparedStatement?
4. What is a ResultSet?
5. Name the JDBC driver types.

Experiment 4 Java Collections Framework

1. Question

Write a Java program to demonstrate key concepts of Collections, such as ArrayList, HashSet, and HashMap.

2. AIM

To demonstrate the usage of ArrayList, HashSet, and HashMap from the Java Collection Framework.

3. Code

```
1 import java.util.*;
2
3 public class CollectionDemo {
4     public static void main(String[] args) {
5         ArrayList<String> list = new ArrayList<>();
6         list.add("Apple");
7         list.add("Banana");
8         list.add("Mango");
9         System.out.println("ArrayList: " + list);
10
11         HashSet<Integer> set = new HashSet<>();
12         set.add(10);
13         set.add(20);
14         set.add(10);
15         System.out.println("HashSet: " + set);
16
17         HashMap<Integer, String> map = new HashMap<>();
18         map.put(1, "One");
19         map.put(2, "Two");
20         map.put(3, "Three");
21         System.out.println("HashMap:");
22         for (Map.Entry<Integer, String> entry : map.entrySet()) {
23             System.out.println(entry.getKey() + " -> " + entry.getValue());
24         }
25     }
26 }
```

4. Output

```
1 ArrayList: [Apple, Banana, Mango]
2 HashSet: [20, 10]
3 HashMap:
4 1 -> One
5 2 -> Two
6 3 -> Three
```

5. Viva Questions

1. What is the Collection Framework in Java?

2. What is the difference between ArrayList and LinkedList?
3. Why does HashSet not allow duplicate elements?
4. How does HashMap store key-value pairs internally?
5. What is the difference between HashMap and TreeMap?

Experiment 5 Integer Division with Exception Handling

1. Question

Write a Java program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a Number Format Exception. If Num2 were Zero, the program would throw an Arithmetic Exception. Display the exception in a message dialog box.

2. AIM

To create a GUI application performing integer division and handling NumberFormatException and ArithmeticException using JOptionPane.

3. Code

```
1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 public class DivisionGUI extends JFrame implements ActionListener {
6     JTextField num1, num2, result;
7     JButton divideBtn;
8
9     DivisionGUI() {
10         setLayout(new GridLayout(4, 2, 5, 5));
11         add(new JLabel("Num1:"));
12         num1 = new JTextField();
13         add(num1);
14
15         add(new JLabel("Num2:"));
16         num2 = new JTextField();
17         add(num2);
18
19         divideBtn = new JButton("Divide");
20         add(divideBtn);
21         divideBtn.addActionListener(this);
22
23         add(new JLabel("Result:"));
24         result = new JTextField();
25         result.setEditable(false);
26         add(result);
27
28         setTitle("Integer Division");
29         setSize(300, 150);
30         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31         setVisible(true);
32     }
33
34     public void actionPerformed(ActionEvent e) {
35         try {
36             int a = Integer.parseInt(num1.getText());
37             int b = Integer.parseInt(num2.getText());
38             int res = a / b;
39             result.setText(String.valueOf(res));
```

```
40         } catch (NumberFormatException ex) {  
41             JOptionPane.showMessageDialog(this, "Please enter valid integers  
!");  
42         } catch (ArithmeticException ex) {  
43             JOptionPane.showMessageDialog(this, "Cannot divide by zero!");  
44         }  
45     }  
46  
47     public static void main(String[] args) {  
48         new DivisionGUI();  
49     }  
50 }
```

4. Output

```
1 Num1 = 10, Num2 = 2 -> Result = 5  
2 Num1 = 10, Num2 = 0 -> Dialog: "Cannot divide by zero!"  
3 Num1 = abc, Num2 = 2 -> Dialog: "Please enter valid integers!"
```

5. Viva Questions

1. What is the difference between `NumberFormatException` and `ArithmeticException`?
2. What is `JOptionPane` used for?
3. Why is exception handling important in GUI applications?
4. What is the parent class of all exceptions in Java?
5. Can we have multiple catch blocks for a single try block? Explain.

Experiment 6 Multithreaded Application

1. Question

Write a Java program that implements a multi-thread application that has three threads. First thread generates a random integer every 1 second and if the value is even, the second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of the cube of the number.

2. AIM

To implement multithreading in Java where multiple threads communicate to compute square or cube of a randomly generated number.

3. Code

```
1 import java.util.Random;
2
3 class NumberGenerator extends Thread {
4     volatile int number;
5     volatile boolean ready = false;
6
7     public void run() {
8         Random rand = new Random();
9         for (int i = 0; i < 5; i++) {
10             number = rand.nextInt(20);
11             ready = true;
12             System.out.println("Generated Number: " + number);
13             try { Thread.sleep(1000); } catch (InterruptedException e) {}
14         }
15     }
16 }
17
18 class SquareThread extends Thread {
19     NumberGenerator gen;
20     SquareThread(NumberGenerator gen) { this.gen = gen; }
21
22     public void run() {
23         for (int i = 0; i < 5; i++) {
24             try { Thread.sleep(1100); } catch (InterruptedException e) {}
25             if (gen.ready && gen.number % 2 == 0) {
26                 System.out.println("Square of " + gen.number + " = " + (gen.
27                     number * gen.number));
28             }
29         }
30     }
31 }
32
33 class CubeThread extends Thread {
34     NumberGenerator gen;
35     CubeThread(NumberGenerator gen) { this.gen = gen; }
36
37     public void run() {
38         for (int i = 0; i < 5; i++) {
39             try { Thread.sleep(1100); } catch (InterruptedException e) {}
40             if (gen.ready && gen.number % 2 != 0) {
41                 System.out.println("Cube of " + gen.number + " = " + (gen.
```

```
        number * gen.number * gen.number));
    }
}
}

46 public class MultiThreadDemo {
47     public static void main(String[] args) {
48         NumberGenerator gen = new NumberGenerator();
49         SquareThread sq = new SquareThread(gen);
50         CubeThread cb = new CubeThread(gen);
51
52         gen.start();
53         sq.start();
54         cb.start();
55     }
56 }
```

4. Output

```
1 Generated Number: 8
2 Square of 8 = 64
3 Generated Number: 15
4 Cube of 15 = 3375
5 Generated Number: 4
6 Square of 4 = 16
7 ...
```

5. Viva Questions

1. What is a thread in Java?
2. What is the difference between extending Thread and implementing Runnable?
3. What does the `volatile` keyword do?
4. What is the purpose of `Thread.sleep()`?
5. What is a race condition, and how can it be avoided?

Experiment 7 ArrayList Insertion and Deletion

1. Question

Write a Java program for the following: Create an array list of elements. Delete a given element from the above list. Display the contents of the list after deletion.

2. AIM

To demonstrate insertion and deletion of elements in an ArrayList.

3. Code

```
1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 public class ArrayListDeleteDemo {
5     public static void main(String[] args) {
6         ArrayList<String> list = new ArrayList<>();
7         list.add("Red");
8         list.add("Green");
9         list.add("Blue");
10        list.add("Yellow");
11
12        System.out.println("Original List: " + list);
13
14        Scanner sc = new Scanner(System.in);
15        System.out.print("Enter element to delete: ");
16        String elem = sc.next();
17
18        if (list.remove(elem)) {
19            System.out.println("Element removed successfully.");
20        } else {
21            System.out.println("Element not found.");
22        }
23
24        System.out.println("List after deletion: " + list);
25    }
26 }
```

4. Output

```
1 Original List: [Red, Green, Blue, Yellow]
2 Enter element to delete: Green
3 Element removed successfully.
4 List after deletion: [Red, Blue, Yellow]
```

5. Viva Questions

1. What is the difference between ArrayList and Array?
2. How does the `remove()` method work in ArrayList?
3. What happens if you try to remove an element that doesn't exist?

4. What is the initial capacity of an ArrayList?
5. Is ArrayList synchronized? How would you make it thread-safe?

Experiment 8 Traffic Light Simulation

1. Question

Write a Java program that simulates a traffic light. The program lets the user select one of three lights: red, yellow, or green with radio buttons. On selecting a button, an appropriate message with "Stop" or "Ready" or "Go" should appear above the buttons in the selected color. Initially, there is no message shown.

2. AIM

To simulate a traffic light system using Java Swing RadioButtons.

3. Code

```
1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 public class TrafficLight extends JFrame implements ActionListener {
6     JRadioButton red, yellow, green;
7     JLabel message;
8
9     TrafficLight() {
10         setTitle("Traffic Light Simulation");
11         setLayout(new BorderLayout());
12
13         message = new JLabel("", SwingConstants.CENTER);
14         message.setFont(new Font("Arial", Font.BOLD, 24));
15         add(message, BorderLayout.NORTH);
16
17         JPanel panel = new JPanel();
18         red = new JRadioButton("Red");
19         yellow = new JRadioButton("Yellow");
20         green = new JRadioButton("Green");
21
22         ButtonGroup group = new ButtonGroup();
23         group.add(red); group.add(yellow); group.add(green);
24
25         panel.add(red); panel.add(yellow); panel.add(green);
26         add(panel, BorderLayout.SOUTH);
27
28         red.addActionListener(this);
29         yellow.addActionListener(this);
30         green.addActionListener(this);
31
32         setSize(350, 150);
33         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34         setVisible(true);
35     }
36
37     public void actionPerformed(ActionEvent e) {
38         if (e.getSource() == red) {
39             message.setText("STOP");
40             message.setForeground(Color.RED);
41         } else if (e.getSource() == yellow) {
42             message.setText("READY");
```

```
43         message.setForeground(Color.ORANGE);
44     } else if (e.getSource() == green) {
45         message.setText("GO");
46         message.setForeground(Color.GREEN);
47     }
48 }
49
50 public static void main(String[] args) {
51     new TrafficLight();
52 }
53 }
```

4. Output

```
1 Selecting "Red" radio button    -> displays "STOP" in red color
2 Selecting "Yellow" radio button -> displays "READY" in orange color
3 Selecting "Green" radio button  -> displays "GO" in green color
```

5. Viva Questions

1. What is a ButtonGroup, and why is it used?
2. What is the difference between JRadioButton and JCheckBox?
3. How does `setForeground()` work?
4. What interface must be implemented to handle radio button clicks?
5. What is the default layout manager of JFrame?

Experiment 9 Abstract Class - Shape Hierarchy

1. Question

Write a Java program to create an abstract class named Shape that contains two integers and an empty method named printArea (). Provide three classes named Rectangle, Triangle, and Circle such that each one of the classes extends the class Shape. Each one of the classes contains only the method printArea () that prints the area of the given shape.

2. AIM

To implement the concept of abstraction using an abstract class Shape with subclasses Rectangle, Triangle, and Circle.

3. Code

```
1 abstract class Shape {
2     int dim1, dim2;
3     Shape(int dim1, int dim2) {
4         this.dim1 = dim1;
5         this.dim2 = dim2;
6     }
7     abstract void printArea();
8 }
9
10 class Rectangle extends Shape {
11     Rectangle(int l, int b) { super(l, b); }
12     void printArea() {
13         System.out.println("Area of Rectangle = " + (dim1 * dim2));
14     }
15 }
16
17 class Triangle extends Shape {
18     Triangle(int b, int h) { super(b, h); }
19     void printArea() {
20         System.out.println("Area of Triangle = " + (0.5 * dim1 * dim2));
21     }
22 }
23
24 class Circle extends Shape {
25     Circle(int r, int dummy) { super(r, dummy); }
26     void printArea() {
27         System.out.println("Area of Circle = " + (3.14 * dim1 * dim1));
28     }
29 }
30
31 public class ShapeDemo {
32     public static void main(String[] args) {
33         Shape s1 = new Rectangle(10, 5);
34         Shape s2 = new Triangle(8, 6);
35         Shape s3 = new Circle(7, 0);
36
37         s1.printArea();
38         s2.printArea();
39         s3.printArea();
40     }
41 }
```

4. Output

```
1 Area of Rectangle = 50
2 Area of Triangle = 24.0
3 Area of Circle = 153.86
```

5. Viva Questions

1. What is an abstract class in Java?
2. Can we instantiate an abstract class? Why or why not?
3. What is the difference between abstract class and interface?
4. Can an abstract class have a constructor?
5. What is runtime polymorphism, and how is it shown here?

Experiment 10 Displaying a Table from a Text File using GridLayout

1. Question

Suppose that a table named Table.txt is stored in a text file. The first line in the file is the header, and the remaining lines correspond to rows in the table. The elements are separated by commas. Write a java program to display the table using Labels in Grid Layout.

2. AIM

To read tabular data from a text file and display it in a GUI using GridLayout and JLabel.

3. Code

Table.txt

```
1 ID,Name,Marks
2 1,John,85
3 2,Alice,90
4 3,Bob,78
```

Java Program

```
1 import java.awt.*;
2 import java.io.*;
3 import java.util.*;
4 import javax.swing.*;
5
6 public class TableDisplay extends JFrame {
7     TableDisplay(String filename) throws IOException {
8         setTitle("Table Display");
9         ArrayList<String[]> rows = new ArrayList<>();
10        BufferedReader br = new BufferedReader(new FileReader(filename));
11        String line;
12        int cols = 0;
13        while ((line = br.readLine()) != null) {
14            String[] parts = line.split(",");
15            cols = parts.length;
16            rows.add(parts);
17        }
18        br.close();
19
20        setLayout(new GridLayout(rows.size(), cols, 5, 5));
21        for (String[] row : rows) {
22            for (String cell : row) {
23                JLabel label = new JLabel(cell, SwingConstants.CENTER);
24                label.setBorder(BorderFactory.createLineBorder(Color.BLACK));
25            }
26            add(label);
27        }
28
29        setSize(400, 200);
30        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31        setVisible(true);
32    }
33
34    public static void main(String[] args) throws IOException {
```

```
35         new TableDisplay("Table.txt");
36     }
37 }
```

4. Output

```
1 A GUI window is displayed as a grid table:
2 -----
3 | ID | Name  | Marks |
4 | 1  | John  | 85    |
5 | 2  | Alice | 90    |
6 | 3  | Bob   | 78    |
7 -----
```

5. Viva Questions

1. What is the use of BufferedReader in file handling?
2. What is GridLayout, and how do rows/columns get defined?
3. What is the difference between FileReader and FileInputStream?
4. How does the `split()` method work in Java?
5. What exceptions can occur while reading a file, and how are they handled?

Experiment 11 Handling Mouse Events using Adapter Classes

1. Question

Write a Java program that handles all mouse events and shows the event name at the center of the window when a mouse event is fired (Use Adapter classes).

2. AIM

To handle mouse events using the MouseAdapter and MouseMotionAdapter classes.

3. Code

```
1 import java.awt.*;
2 import java.awt.event.*;
3 import javax.swing.*;
4
5 public class MouseEventsDemo extends JFrame {
6     JLabel label;
7
8     MouseEventsDemo() {
9         setTitle("Mouse Events Demo");
10        label = new JLabel("No Event", SwingConstants.CENTER);
11        label.setFont(new Font("Arial", Font.BOLD, 20));
12        add(label);
13
14        addMouseListener(new MouseAdapter() {
15            public void mouseClicked(MouseEvent e) { label.setText("Mouse
Clicked"); }
16            public void mousePressed(MouseEvent e) { label.setText("Mouse
Pressed"); }
17            public void mouseReleased(MouseEvent e) { label.setText("Mouse
Released"); }
18            public void mouseEntered(MouseEvent e) { label.setText("Mouse
Entered"); }
19            public void mouseExited(MouseEvent e) { label.setText("Mouse
Exited"); }
20        });
21
22        addMouseMotionListener(new MouseMotionAdapter() {
23            public void mouseMoved(MouseEvent e) { label.setText("Mouse
Moved"); }
24            public void mouseDragged(MouseEvent e) { label.setText("Mouse
Dragged"); }
25        });
26
27        setSize(400, 300);
28        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29        setVisible(true);
30    }
31
32    public static void main(String[] args) {
33        new MouseEventsDemo();
34    }
35 }
```

4. Output

1	Moving mouse inside window	-> "Mouse Moved "
2	Clicking inside window	-> "Mouse Clicked "
3	Dragging mouse	-> "Mouse Dragged "
4	Mouse entering the window frame	-> "Mouse Entered "
5	Mouse leaving the window frame	-> "Mouse Exited "

5. Viva Questions

1. What is an Adapter class in Java? Why is it used?
2. Difference between MouseListener and MouseMotionListener?
3. What are the methods declared in the MouseListener interface?
4. Why do we prefer Adapter classes over implementing full interfaces?
5. What is event delegation model in Java?

Experiment 12 Contact Lookup using Hash Table

1. Question

Write a Java program that loads names and phone numbers from a text file where the data is organized as one line per record and each field in a record is separated by a tab (\t). It takes a name or phone number as input and prints the corresponding other value from the hash table (hint: use hash tables).

2. AIM

To read data from a text file into a Hashtable and perform a lookup by name or phone number.

3. Code

contacts.txt

```
1 John 9876543210
2 Alice 9123456789
3 Bob 9988776655
```

Java Program

```
1 import java.io.*;
2 import java.util.*;
3
4 public class ContactLookup {
5     public static void main(String[] args) throws IOException {
6         Hashtable<String, String> nameToPhone = new Hashtable<>();
7         Hashtable<String, String> phoneToName = new Hashtable<>();
8
9         BufferedReader br = new BufferedReader(new FileReader("contacts.txt"
10    ));
11         String line;
12         while ((line = br.readLine()) != null) {
13             String[] parts = line.split("\t");
14             nameToPhone.put(parts[0], parts[1]);
15             phoneToName.put(parts[1], parts[0]);
16         }
17         br.close();
18
19         Scanner sc = new Scanner(System.in);
20         System.out.print("Enter name or phone number: ");
21         String input = sc.next();
22
23         if (nameToPhone.containsKey(input)) {
24             System.out.println("Phone Number: " + nameToPhone.get(input));
25         } else if (phoneToName.containsKey(input)) {
26             System.out.println("Name: " + phoneToName.get(input));
27         } else {
28             System.out.println("Record not found.");
29         }
30     }
31 }
```

4. Output

```
1 Enter name or phone number: John
```

```
2 Phone Number: 9876543210
3
4 Enter name or phone number: 9123456789
5 Name: Alice
```

5. Viva Questions

1. What is a Hashtable? How is it different from HashMap?
2. Why is Hashtable synchronized while HashMap is not?
3. What is the significance of hashCode() and equals() methods?
4. Can Hashtable store null keys or values?
5. What is the time complexity of get() and put() operations in a Hashtable?

Experiment 13 Producer-Consumer Problem

1. Question

Write a Java program that correctly implements the producer - consumer problem using the concept of inter-thread communication.

2. AIM

To implement the Producer-Consumer problem using wait() and notify() for inter-thread communication.

3. Code

```
1 import java.util.LinkedList;
2
3 class SharedBuffer {
4     LinkedList<Integer> list = new LinkedList<>();
5     int capacity = 5;
6
7     public synchronized void produce(int value) throws InterruptedException
8     {
9         while (list.size() == capacity) {
10             wait();
11         }
12         list.add(value);
13         System.out.println("Produced: " + value);
14         notify();
15     }
16
17     public synchronized void consume() throws InterruptedException {
18         while (list.isEmpty()) {
19             wait();
20         }
21         int value = list.removeFirst();
22         System.out.println("Consumed: " + value);
23         notify();
24     }
25 }
26
27 class Producer extends Thread {
28     SharedBuffer buffer;
29     Producer(SharedBuffer buffer) { this.buffer = buffer; }
30     public void run() {
31         for (int i = 1; i <= 10; i++) {
32             try {
33                 buffer.produce(i);
34                 Thread.sleep(300);
35             } catch (InterruptedException e) {}
36         }
37     }
38 }
39
40 class Consumer extends Thread {
41     SharedBuffer buffer;
42     Consumer(SharedBuffer buffer) { this.buffer = buffer; }
43     public void run() {
```

```
43         for (int i = 1; i <= 10; i++) {  
44             try {  
45                 buffer.consume();  
46                 Thread.sleep(500);  
47             } catch (InterruptedException e) {}  
48         }  
49     }  
50 }  
51  
52 public class ProducerConsumerDemo {  
53     public static void main(String[] args) {  
54         SharedBuffer buffer = new SharedBuffer();  
55         Producer p = new Producer(buffer);  
56         Consumer c = new Consumer(buffer);  
57         p.start();  
58         c.start();  
59     }  
60 }
```

4. Output

```
1 Produced: 1  
2 Consumed: 1  
3 Produced: 2  
4 Produced: 3  
5 Consumed: 2  
6 Produced: 4  
7 Consumed: 3  
8 ...
```

5. Viva Questions

1. What is the Producer-Consumer problem?
2. What is the role of wait() and notify() in Java?
3. Why must wait() and notify() be called within a synchronized block?
4. What is a deadlock, and how is it avoided here?
5. What is the difference between notify() and notifyAll()?

Experiment 14 Recursively Listing Files in a Directory

1. Question

Write a Java program to list all the files in a directory including the files present in all its subdirectories.

2. AIM

To recursively list all files in a given directory and its subdirectories using Java's File class.

3. Code

```
1 import java.io.File;
2
3 public class ListFilesDemo {
4     public static void listFiles(File dir) {
5         File[] files = dir.listFiles();
6         if (files != null) {
7             for (File file : files) {
8                 if (file.isDirectory()) {
9                     System.out.println("[DIR] " + file.getAbsolutePath());
10                    listFiles(file);
11                } else {
12                    System.out.println(file.getAbsolutePath());
13                }
14            }
15        }
16    }
17
18    public static void main(String[] args) {
19        File dir = new File("C:/SampleFolder");
20        if (dir.exists() && dir.isDirectory()) {
21            listFiles(dir);
22        } else {
23            System.out.println("Directory does not exist.");
24        }
25    }
26 }
```

4. Output

```
1 [DIR] C:/SampleFolder/Docs
2 C:/SampleFolder/Docs/resume.pdf
3 C:/SampleFolder/Docs/notes.txt
4 [DIR] C:/SampleFolder/Images
5 C:/SampleFolder/Images/photo.jpg
6 C:/SampleFolder/readme.txt
```

5. Viva Questions

1. What is the File class used for in Java?
2. What is the difference between `listFiles()` and `list()`?

3. How does recursion help in traversing a directory tree?
4. How would you filter only specific file types (e.g., .txt) in this program?
5. What is the difference between absolute path and relative path?